CAPSTONE 4

SMART HOME IoT PROJECT DOCUMENTATION

AI-Assisted Planning, Design, Bill of Materials, Code, Testing and Presentation

Project title	Smart Room Temperature and Humidity Monitoring System
Controller	ESP32 DevKit V1
Sensor	DHT22 / AM2302
Display	0.96-inch SSD1306 I2C OLED
Prepared	30 October 2026
Student details	Name: _____ Class / Department: _____

A student prototype for observing indoor conditions and giving a local visual alert when a configurable threshold is crossed.

Document guide

This document is designed as a complete project pack. It contains a build plan, wiring reference, sample firmware, verification plan, AI prompt log, automation example, README/user guide and a six-slide presentation outline.

Important evidence note: The code and logic are provided for implementation. The test table includes simulated input cases for checking the decision logic; it is not proof that a physical ESP32 was flashed or that a Wokwi simulation was executed. Before submission, run the sketch in Wokwi or on the actual board, save screenshots, and complete the student evidence checklist.

Contents

• 1. Executive summary and objectives

• 2. Requirements and system design

• 3. Block diagram and operating logic

• 4. Component research and verified BOM

• 5. Wiring and pin connections

• 6. Firmware and setup instructions

• 7. Test plan, sample simulation and verification log

• 8. AI tools, prompts and correction record

• 9. BOM automation script

• 10. Safety, privacy and limitations

• 11. README and user guide

• 12. Poster concept and six-slide presentation

• 13. Final submission checklist and references

1. Executive summary and objectives

The Smart Room Temperature and Humidity Monitoring System measures indoor air temperature and relative humidity. A DHT22 sensor sends digital readings to an ESP32 microcontroller. The ESP32 displays the values on a small OLED screen and switches on an LED when temperature or humidity reaches a configurable alert threshold.

Problem statement

People may want a quick indication of room conditions without opening a phone app or checking a separate instrument. This prototype provides a simple local display and visual alert.

Objectives

• Measure room temperature in degrees Celsius.

• Measure relative humidity as a percentage.

• Display readings locally on an OLED.

• Turn on an LED when temperature is at least 30 °C or humidity is at least 70% RH.

• Show an error message if the sensor reading is invalid.

• Keep the design low-cost, understandable and expandable to Wi-Fi logging later.

Scope

This version is a local monitor. It does not upload readings to a cloud service, identify people, control an air conditioner, or provide medical/health advice. Thresholds are demonstration values and can be changed in the code.

Success criteria

The system is considered functionally successful when valid sensor readings appear on the OLED and serial monitor, the LED remains off below both thresholds, the LED turns on when either threshold is crossed, and invalid readings are handled without displaying misleading values.

2. Requirements and system design

Functional requirements

ID	Requirement	Acceptance check
FR-01	Read DHT22 temperature and humidity every 2.5 seconds.	Fresh values appear periodically.
FR-02	Show temperature and humidity on OLED.	Both labels and units are visible.
FR-03	Alert if temperature ≥ 30.0 °C OR humidity ≥ 70.0% RH.	LED turns ON when either condition is true.
FR-04	Handle missing/invalid sensor data.	Error message appears; LED is OFF.
FR-05	Print readings and status to serial monitor.	Readable serial log at 115200 baud.

Non-functional requirements

• Low-cost student prototype using commonly available modules.

• Readable display and simple wiring.

• Use 3.3 V logic-compatible connections on ESP32 GPIO pins.

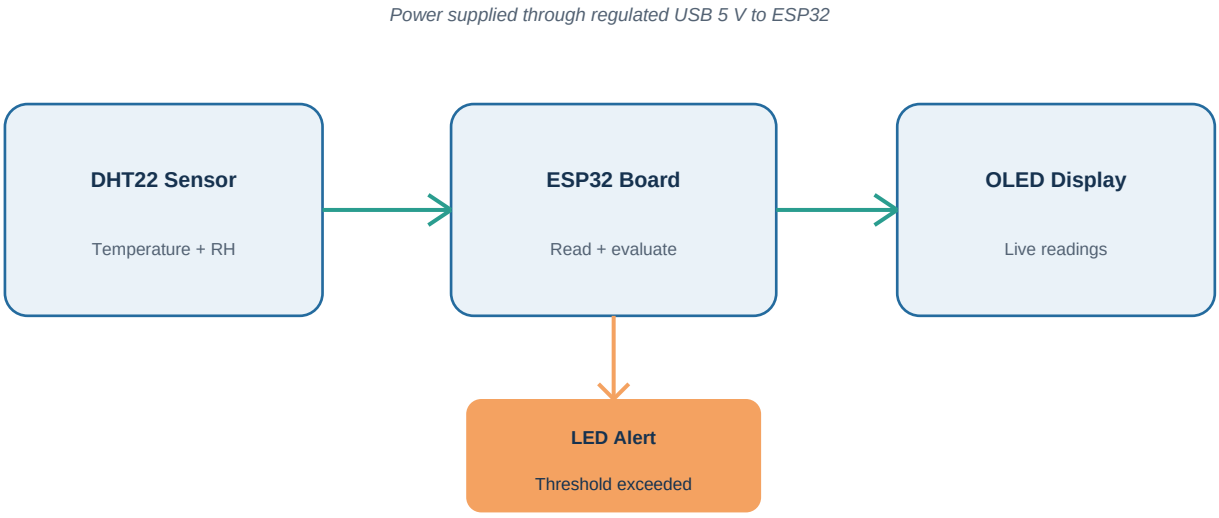
• Firmware should continue to show a clear error state if sensor data fails.

• No personal data is collected or transmitted in this local-only version.

System architecture

The DHT22 is the input device. ESP32 reads and checks the values. OLED is the output display. An LED is the local warning output. USB supplies power and is also used for programming.

3. Block diagram and operating logic



Operating sequence

• Power on the ESP32 using a USB cable.

• Initialize the DHT22, I2C bus, OLED and alert LED.

• Wait for the 2.5-second measurement interval.

• Read temperature and humidity from the sensor.

• If either value is invalid, display a sensor error and keep the LED OFF.

• Otherwise compare the values with the configured thresholds.

• Display the readings and status; switch the LED ON for an alert and OFF for normal conditions.

• Repeat continuously.

Decision rule

Alert = (Temperature ≥ 30.0 °C) OR (Humidity ≥ 70.0% RH)
The OR condition means either high temperature or high humidity is sufficient to activate the LED. Adjust these demo thresholds to suit the classroom test.

4. Component research and verified BOM

Component selection rationale

ESP32 DevKit V1 was selected because it includes Wi-Fi/Bluetooth capability and supports Arduino IDE development. DHT22 was selected for digital temperature and humidity readings. The SSD1306 OLED provides a compact I2C display using only two signal wires. The LED gives an easy-to-see local alert.

Vendor price check (30 October 2026)

The first three prices below were visible on Indian vendor product listings during research. Prices and stock can change; confirm the exact board variant, shipping and tax at checkout.

Component	Qty	Unit price (INR)	Line total	Price status / source
ESP32 DevKit V1, 30-pin	1	419	419	Listed price; Robosix Lab
DHT22 / AM2302 module	1	129	129	Listed price; AM Electronics
0.96-inch SSD1306 I2C OLED	1	188	188	Listed price; TecnoMate
Breadboard	1	80	80	Planning estimate
Male-female jumper wires	1 set	60	60	Planning estimate
LED + 220 Ω resistor	1 set	10	10	Planning estimate
USB data cable	1	80	80	Planning estimate
TOTAL			966	Estimated total; excludes shipping

Vendor pages used for the three priced components

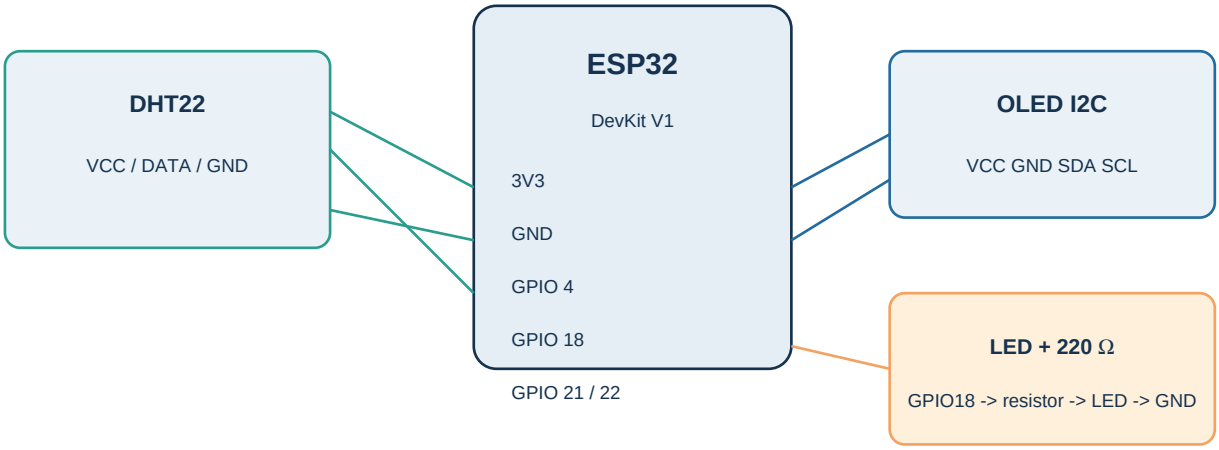
ESP32 DevKit V1 — Robosix Lab: https://robosixlab.com/shop/esp32-devkit-v1-wifi-and-bluetooth-board-30-pin

DHT22 module — AM Electronics: https://amelectronics.in/product/dht22-digital-temperature-humidity-sensor-am2302/

0.96-inch OLED — TecnoMate: https://tecnomate.in/component-library/0-96-inch-oled-display-module-i2c-ssd1306

Cross-check: alternative listings include ESP32 DevKit V1 at Partsly and DHT22 at KSP Electronics. Vendor listings may refer to different variants or stock conditions; compare before purchasing.

5. Wiring and pin connections



Connection table

Module pin	ESP32 connection	Notes
DHT22 VCC	3V3	Use 3.3 V supply.
DHT22 DATA	GPIO 4	If using a bare 4-pin sensor, add a 4.7–10 kΩ pull-up from DATA to 3V3. Many breakout modules already include it.
DHT22 GND	GND	Common ground.
OLED VCC	3V3	Check your module's voltage specification.
OLED GND	GND	Common ground.
OLED SDA	GPIO 21	I2C data.
OLED SCL	GPIO 22	I2C clock.
LED anode (+)	GPIO 18 through 220 Ω resistor	Resistor must be in series.
LED cathode (–)	GND	Observe LED polarity.

Manual wiring checks: Disconnect USB power before changing wires. Verify that 3V3 and GND are not shorted. Confirm the ESP32 pin labels from the exact board pinout. Do not connect a 5 V signal to an ESP32 GPIO. Check the OLED I2C address (commonly 0x3C; some modules use 0x3D).

6. Firmware and setup instructions

Software requirements

Install Arduino IDE and add ESP32 board support using Espressif's official instructions. In Library Manager install: DHT sensor library by Adafruit, Adafruit Unified Sensor, Adafruit GFX Library and Adafruit SSD1306.

Official setup reference: https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

Adafruit DHT library and dependency information: https://github.com/adafruit/DHT-sensor-library

Adafruit DHT sensor guide: https://learn.adafruit.com/dht/using-a-dhtxx-sensor-with-arduino

Upload procedure

• Open Arduino IDE and create a new sketch.

• Paste the complete firmware from Appendix A.

• Select the matching ESP32 Dev Module board and the correct serial port.

• Compile the sketch. Resolve missing-library errors by installing the named libraries.

• Connect the circuit and upload. If the board does not enter upload mode, follow the board manufacturer's instructions; board variants differ.

• Open Serial Monitor at 115200 baud and observe the readings.

• Check the OLED display and test the alert threshold in a controlled, safe way.

Appendix A — Complete Arduino sketch

The code below is the sample firmware. It uses an update interval of 2.5 seconds because the DHT22 should not be polled rapidly.

```
/*
  Smart Room Temperature & Humidity Monitor
  Board: ESP32 DevKit V1
  Sensor: DHT22 on GPIO 4
  OLED: SSD1306 I2C, SDA=21, SCL=22
  Alert LED: GPIO 18 through a 220-ohm resistor
  Libraries: DHT sensor library, Adafruit Unified Sensor,
             Adafruit GFX, Adafruit SSD1306
*/
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

#define DHT_PIN 4
#define DHT_TYPE DHT22
#define LED_PIN 18
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

const float TEMP_LIMIT_C = 30.0;
const float HUMIDITY_LIMIT = 70.0;
const unsigned long READ_INTERVAL_MS = 2500;

DHT dht(DHT_PIN, DHT_TYPE);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
unsigned long lastRead = 0;

void showError() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 12);
  display.println("Sensor read error");
  display.println("Check DHT22 wiring");
  display.display();
  digitalWrite(LED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Wire.begin(21, 22);
  dht.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found. Check I2C wiring/address.");
```

```
    while (true) delay(1000);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 10);
  display.println("Smart Room Monitor");
  display.println("Starting...");
  display.display();
  delay(1500);
}

void loop() {
  if (millis() - lastRead < READ_INTERVAL_MS) return;
  lastRead = millis();

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature(); // Celsius

  if (isnan(humidity) || isnan(temperature)) {
    showError();
    Serial.println("ERROR: invalid DHT22 reading");
    return;
  }

  bool alert = (temperature >= TEMP_LIMIT_C) ||
               (humidity >= HUMIDITY_LIMIT);
  digitalWrite(LED_PIN, alert ? HIGH : LOW);

  Serial.print("Temperature: ");
  Serial.print(temperature, 1);
  Serial.print(" C | Humidity: ");
  Serial.print(humidity, 1);
  Serial.print(" % | Status: ");
  Serial.println(alert ? "ALERT" : "NORMAL");

  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("SMART ROOM MONITOR");
  display.drawLine(0, 11, 127, 11, SSD1306_WHITE);
  display.setCursor(0, 18);
  display.print("Temp: ");
  display.print(temperature, 1);
  display.println(" C");
  display.setCursor(0, 32);
  display.print("Humidity: ");
  display.print(humidity, 1);
  display.println(" %");
  display.setCursor(0, 49);
  display.print("Status: ");
  display.println(alert ? "CHECK ROOM" : "NORMAL");
  display.display();
}
```

7. Test plan, sample simulation and verification log

Test cases

Test	Input / condition	Expected result
T1	24.0 °C, 50% RH	Normal status; LED OFF.
T2	31.0 °C, 45% RH	Alert status; LED ON.
T3	27.0 °C, 75% RH	Alert status; LED ON.
T4	30.0 °C, 70% RH	Alert status; LED ON (inclusive threshold).
T5	Invalid / NaN sensor reading	Sensor error displayed; LED OFF.
T6	Readings below both thresholds	OLED and serial show valid values; LED OFF.

Logic-only simulated test results

The following results are calculated from the stated Boolean rule. They validate the decision logic on paper; they are not measurements from a real DHT22 or a screenshot from Wokwi.

Case	Temp	RH	Alert expression	Expected LED
T1	24.0	50	false OR false	OFF
T2	31.0	45	true OR false	ON
T3	27.0	75	false OR true	ON
T4	30.0	70	true OR true	ON
T5	NaN	NaN	invalid data branch	OFF

Actual verification record — complete after running

Date/time: _____ Board model: _____ Arduino IDE version: _____ Wokwi / physical board: _____

Compilation result: _____ Upload/simulation result: _____ OLED address found: _____

Attach evidence: (1) circuit view, (2) serial monitor normal reading, (3) alert LED ON case, (4) sensor-error or threshold test if available.

Common issues and corrections

• OLED blank: verify SDA=21, SCL=22, supply and I2C address (0x3C/0x3D).

• DHT invalid values: confirm DHT22 type, DATA pin, common GND and pull-up resistor if required.

• Compile error: install all four libraries and their dependencies.

• LED never turns on: test values against ≥ thresholds; verify LED polarity and GPIO18 wiring.

• ESP32 upload failure: confirm USB data cable, selected port and board variant.

8. AI tools, prompts and correction record

AI-assisted workflow record

This project document was drafted with ChatGPT assistance. The following second-tool cross-check prompts are supplied for use in Gemini or Perplexity. To satisfy a strict requirement for two AI research tools, run at least one prompt in a second tool, record its response/date, and compare its result with the vendor datasheets and product pages. Do not claim a second tool was used until you have actually run it.

Tool 1 — ChatGPT (used for planning and drafting)

Prompt 1: “Suggest a beginner-friendly ESP32 IoT project that measures room temperature and humidity. List requirements, architecture, and testable acceptance criteria.”

Prompt 2: “Review this ESP32 + DHT22 + SSD1306 design. Check pin mapping, logic thresholds, error handling, and possible electrical mistakes. State assumptions.”

Prompt 3: “Generate an Arduino sketch using DHT22 on GPIO4, SSD1306 I2C on GPIO21/22, and an alert LED on GPIO18. Include invalid-reading handling.”

Tool 2 — Gemini or Perplexity (student cross-check to complete)

Prompt 4: “Independently research whether ESP32 DevKit V1, DHT22 and SSD1306 I2C OLED are compatible at 3.3 V logic. Give authoritative datasheet or vendor references and identify any caveats.”

Prompt 5: “Review this bill of materials and identify missing components, price assumptions, and any wiring risks. Do not invent vendor prices; link to source pages.”

Prompt 6: “Create test cases for the rule temperature ≥30 °C OR humidity ≥70% RH, including invalid sensor data and boundary values.”

Prompt 7: “Review the README and user guide for clarity and safety. List corrections without changing hardware assumptions.”

AI error and correction log

AI suggestion / possible issue	Verification method	Correction / decision
Could assume OLED address is always 0x3C.	Check module label / I2C scanner.	Code uses 0x3C; change to 0x3D if actual module requires it.
Could omit DHT22 pull-up detail.	Check sensor module type and datasheet.	Add 4.7–10 kΩ pull-up only if bare sensor/module lacks it.
Could treat logic test as hardware proof.	Compare evidence type.	Report labels it logic-only; physical/Wokwi test evidence remains to be captured.
Could provide stale prices.	Open live vendor product pages.	Prices are date-stamped and non-priced items are marked estimates.

9. BOM automation script

Automation task: convert a simple component CSV into a formatted BOM CSV with serial numbers and line totals. This demonstrates one repeatable project step. The script uses Python's built-in csv module.

Input file example — components.csv

```
Component,Specification,Quantity,Unit Price (INR),Price Type,Vendor / Source\nESP32 DevKit V1,30-pin board,1,419,Listed,Robosix
```

Automation script — format_bom.py

```
import csv

INPUT_FILE = "components.csv"
OUTPUT_FILE = "formatted_BOM.csv"

with open(INPUT_FILE, newline="", encoding="utf-8") as source:
    rows = list(csv.DictReader(source))

for index, row in enumerate(rows, start=1):
    row["S.No"] = str(index)
    row["Line Total (INR)"] = str(
        round(float(row["Unit Price (INR)"]) * int(row["Quantity"]), 2)
    )

fields = ["S.No", "Component", "Specification", "Quantity",
          "Unit Price (INR)", "Line Total (INR)", "Price Type", "Vendor / Source"]

with open(OUTPUT_FILE, "w", newline="", encoding="utf-8") as target:
    writer = csv.DictWriter(target, fieldnames=fields)
    writer.writeheader()
    writer.writerows(rows)

print("Formatted BOM saved to", OUTPUT_FILE)
```

Run: python format_bom.py. The script creates formatted_BOM.csv in the same folder. Check that quantities and prices are numeric before running. Keep the original vendor URLs in the project evidence folder.

10. Safety, data privacy and limitations

Electrical safety

• Use only the USB supply recommended for the development board; inspect the cable and board before powering.

• Disconnect power before changing connections.

• Never short 3V3 to GND. Confirm polarity and pin labels before connecting modules.

• ESP32 GPIO uses 3.3 V logic. Do not feed 5 V into GPIO pins.

• Use a series resistor with the external LED. Keep the prototype away from liquids, heat sources and metal objects that can short pins.

• Ask a teacher or knowledgeable adult to check the circuit before first power-up.

Privacy and security

The current design is local-only: it does not store names, audio, images, location or personal information, and it does not connect to Wi-Fi. If cloud features are added later, use a unique account, strong credentials, encrypted connections where supported, minimal data collection, and clear permission from the people affected. Never publish Wi-Fi passwords, API keys or personal information in code screenshots.

Limitations

• DHT22 is a low-cost sensor and should not be treated as a calibrated laboratory instrument.

• Sensor placement, airflow, sunlight and nearby electronics can affect readings.

• The alert thresholds are demonstration values, not universal comfort or safety limits.

• No remote notifications, data history, battery backup or enclosure are included.

• The OLED address and ESP32 pin layout may differ between board/module variants.

• A prototype is not a certified safety, HVAC or medical device.

Future improvements

Add Wi-Fi dashboard, MQTT/ThingSpeak logging, a real-time clock, data export, a proper enclosure, and calibration checks. Each addition needs separate privacy and security review.

11. README and user guide

README.md (project repository text)

```
# Smart Room Temperature and Humidity Monitor

## Overview
ESP32 reads a DHT22 sensor and shows temperature and relative
humidity on a 0.96-inch SSD1306 I2C OLED. An LED turns on when
temperature >= 30 C OR humidity >= 70% RH.

## Hardware
- ESP32 DevKit V1
- DHT22 / AM2302
- SSD1306 I2C OLED (128x64)
- LED, 220-ohm resistor, breadboard, jumper wires
- USB data cable

## Pin map
DHT22 DATA -> GPIO 4
OLED SDA -> GPIO 21; SCL -> GPIO 22
LED -> GPIO 18 through 220-ohm resistor
All modules share GND. Use 3.3 V for sensor and OLED if supported.

## Libraries
DHT sensor library by Adafruit
Adafruit Unified Sensor
Adafruit GFX Library
Adafruit SSD1306

## Build
Install ESP32 board support in Arduino IDE, install libraries,
open the sketch, select the correct ESP32 board and port, compile,
upload, then open Serial Monitor at 115200 baud.

## Thresholds
Edit TEMP_LIMIT_C and HUMIDITY_LIMIT near the top of the sketch.

## Safety
Disconnect USB before rewiring. Do not apply 5 V to ESP32 GPIO.
This is an educational prototype, not a certified safety device.
```

Quick user guide

1. Place the sensor in the room, away from direct sunlight, water and hot appliances.
2. Connect the ESP32 to a USB power source.
3. Wait for the startup message to finish.
4. Read temperature (°C) and humidity (% RH) on the OLED.
5. If the status says CHECK ROOM and the LED is ON, one of the demo thresholds has been reached.
6. If Sensor read error appears, switch off power and check the wiring and sensor connection.
7. To change thresholds, edit the constants in the sketch and upload again.

Maintenance

Keep the sensor dry and dust-free. Recheck connections if readings stop. Do not expose the board to condensation or liquids. Record the date and conditions when collecting demonstration readings.

12. Poster concept and six-slide presentation

Poster image concept 1 — project overview

SMART ROOM MONITOR
ESP32 + DHT22 + OLED

MEASURE → PROCESS → DISPLAY → ALERT

Temperature: °C
Humidity: % RH
Alert: LED ON when either demo threshold is crossed

Low-cost • Local display • Beginner-friendly • Expandable

Suggested visual assets to create for the final submission

1. Block diagram (page 3). 2. Wiring diagram (page 5). 3. OLED screen mock-up showing sample values. 4. A project poster using the text above. 5. A photo or screenshot of the completed Wokwi/physical prototype. The fifth image must be captured from the actual build; do not present a mock-up as a real prototype.

Six-slide presentation storyboard

Slide 1 — Title

Smart Room Temperature and Humidity Monitor | CAPSTONE 4 | Student name and class.

Slide 2 — Problem and objective

Why monitor indoor conditions? Measure temperature and humidity and provide a local visual alert.

Slide 3 — Hardware and BOM

ESP32, DHT22, SSD1306 OLED, LED, resistor, breadboard and jumper wires. Show estimated cost: ■966.

Slide 4 — Architecture and wiring

Show block diagram and pin mapping. Explain sensor input, ESP32 processing, OLED output and LED alert.

Slide 5 — Code and testing

Explain 2.5-second sampling, thresholds and invalid-data handling. Show test cases and actual simulation screenshots.

Slide 6 — Results, limitations and future work

Summarize observed results after testing, note sensor accuracy/placement limits, privacy and future Wi-Fi dashboard.

13. Final submission checklist and references

Before submitting

- ■ Add student name, department, register number and institution details on the cover if required.
- ■ Run the code in Wokwi or on the actual ESP32. Capture circuit, serial monitor and OLED/alert evidence.
- ■ Run a second AI research tool (Gemini or Perplexity) and save the prompt, date and response summary.
- ■ Open each vendor page again and update the price and stock status on the submission date.
- ■ Manually compare every connection with the exact ESP32 and module pinout.
- ■ Export the six-slide storyboard into a presentation file if your teacher requires a separate PPT.
- ■ Attach the generated formatted BOM CSV and any automation run screenshot.
- ■ Have the wiring and safety checked by a teacher before powering the physical circuit.

References

1. Robosix Lab, ESP32 DevKit V1 product listing: https://robosixlab.com/shop/esp32-devkit-v1-wifi-and-bluetooth-board-30-pin

2. AM Electronics, DHT22 / AM2302 sensor listing: https://amelectronics.in/product/dht22-digital-temperature-humidity-sensor-am2302/

3. TecnoMate, SSD1306 OLED listing: https://tecnomate.in/component-library/0-96-inch-oled-display-module-i2c-ssd1306

4. Espressif, Arduino-ESP32 installation documentation: https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

5. Adafruit, DHT sensor library: https://github.com/adafruit/DHT-sensor-library

6. Adafruit, Using a DHTxx sensor with Arduino: https://learn.adafruit.com/dht/using-a-dhtxx-sensor-with-arduino

7. Alternative cross-check listings: https://partsly.in/products/esp32-devkit-v1-30pin and https://kspelectronics.in/product/dht22-digital-temperature-and-humidity-sensor-module/

Research note: Vendor prices were observed during preparation on 30 October 2026. Retail listings are dynamic and do not guarantee availability, delivery cost or exact module revision.

END OF PROJECT DOCUMENT